



Department of Computer Science and Engineering
2024 CSE BATCH
23CSE463 – Quantum Computing

Agneay B Nair
CH.SC.U4CSE24102

WORKSHEET



WORKSHEET- 3

EX.NO : 2

DATE : 16/7/2026

TITLE : Matrix Representation, Verification, and Simulation of Fundamental Single-Qubit Quantum Gates Using PennyLane

AIM:

To study the fundamentals of the PennyLane quantum computing framework and understand its architecture for designing and executing quantum circuits. The objective is to create quantum devices, implement QNodes, apply basic single-qubit quantum gates, perform quantum measurements, and analyze the results using PennyLane, thereby providing a foundation for developing hybrid quantum-classical algorithms and quantum machine learning applications.

APPARATUS REQUIRED:

- Python 3.10 or above
- Qiskit Quantum Computing Framework
- Jupyter Notebook / Visual Studio Code (VS Code)
- Internet Connection (for package installation and documentation access)
- Quantum Computing Reference Material or Lab Manual

THEORY:

PennyLane is an open-source quantum computing software framework developed by Xanadu for designing, simulating, and optimizing quantum circuits. It enables seamless integration of quantum computing with classical machine learning libraries such as PyTorch, TensorFlow, and JAX, making it a powerful platform for hybrid quantum-classical computing.

• Quantum Devices

A quantum device represents either a quantum simulator or real quantum hardware on which quantum circuits are executed. In PennyLane, quantum devices are created using the `qml.device()` function, where the number of qubits (wires) and the execution backend are specified.



- **Quantum Nodes (QNodes)**

A Quantum Node (QNode) is the fundamental computational unit in PennyLane. It combines a quantum circuit with a quantum device, allowing quantum operations to be executed and measurement results to be returned to classical programs. QNodes are created using the `@qml.qnode` decorator.

- **Quantum Gates**

Quantum gates are unitary operations that manipulate the state of qubits. PennyLane provides a wide range of built-in quantum gates such as Pauli-X, Pauli-Y, Pauli-Z, Hadamard, Phase Shift, and Rotation gates for constructing quantum circuits.

- **Quantum Measurements**

After executing a quantum circuit, measurements are performed to extract information from the qubits. PennyLane supports various measurement operations, including expectation values, probabilities, quantum states, and sample measurements.

- **Hybrid Quantum-Classical Computing**

One of the key advantages of PennyLane is its ability to combine quantum circuits with classical optimization algorithms. This hybrid approach enables the development of variational quantum algorithms and quantum machine learning models for solving complex computational problems.

- **Applications of PennyLane**

PennyLane is widely used in quantum machine learning, quantum optimization, variational quantum algorithms, quantum chemistry, and quantum neural network research. Its flexibility and compatibility with multiple quantum hardware platforms make it an important tool for quantum computing education and research.

PROCEDURE:

1. Install the PennyLane framework and the required Python libraries.
2. Open Jupyter Notebook or Visual Studio Code.
3. Import the required PennyLane modules.
4. Create a quantum device using the `qml.device()` function.
5. Define a Quantum Node (QNode) using the `@qml.qnode` decorator.
6. Construct a simple quantum circuit by applying single-qubit quantum gates.
7. Execute the quantum circuit on the selected quantum device.



8. Perform quantum measurements such as state vectors, probabilities, and expectation values.
9. Visualize and analyze the quantum circuit and its measurement results.
10. Observe and verify the implementation of quantum circuits using the PennyLane framework.

PROGRAM:

Program 1: Implement the Identity gate matrix and verify its operation on the computational basis states $|0\rangle$ and $|1\rangle$ using matrix multiplication and PennyLane.

OUTPUT:

Code:

```
dev = qml.device('default.qubit', wires=1)
ket0 = np.array([1, 0], dtype=complex)
ket1 = np.array([0, 1], dtype=complex)

I = np.array([[1, 0], [0, 1]], dtype=complex)
print("Identity matrix I =\n", I)

def run_gate(gate_matrix, state, gate_op):
    classical = gate_matrix @ state
    @qml.qnode(dev)
    def circuit():
        qml.StatePrep(state, wires=0)
        gate_op(wires=0)
        return qml.state()
    return classical, circuit()

for label, state in [("|0>", ket0), ("|1>", ket1)]:
    c, q = run_gate(I, state, qml.Identity)
    print(f"I . {label} (matrix)    = {c}")
    print(f"I . {label} (PennyLane) = {q}")
```

Output:

```
Identity matrix I =
[[1.+0.j 0.+0.j]
 [0.+0.j 1.+0.j]]
I . |0> (matrix)    = [1.+0.j 0.+0.j]
I . |0> (PennyLane) = [1.+0.j 0.+0.j]
I . |1> (matrix)    = [0.+0.j 1.+0.j]
I . |1> (PennyLane) = [0.+0.j 1.+0.j]
```

Observation: The Identity gate leaves both basis states completely unchanged.

Program 2: Implement the Pauli-X Gate Matrix and Verify Its Transformation.

OUTPUT:

Code:

```
X = np.array([[0, 1], [1, 0]], dtype=complex)
print("Pauli-X matrix X =\n", X)
```



```
for label, state in [("|0>", ket0), ("|1>", ket1)]:
    c, q = run_gate(X, state, qml.PauliX)
    print(f"X . {label} (matrix)    = {c}")
    print(f"X . {label} (PennyLane) = {q}")
```

Output:

```
Pauli-X matrix X =
[[0.+0.j 1.+0.j]
 [1.+0.j 0.+0.j]]
X . |0> (matrix)    = [0.+0.j 1.+0.j]
X . |0> (PennyLane) = [0.+0.j 1.+0.j]
X . |1> (matrix)    = [1.+0.j 0.+0.j]
X . |1> (PennyLane) = [1.+0.j 0.+0.j]
```

Observation: Pauli-X acts as a classical bit-flip (NOT) gate: $|0\rangle \leftrightarrow |1\rangle$.

Program 3: Implement the Pauli-Y Gate Matrix and Analyze the Phase Transformation.

OUTPUT:

Code:

```
Y = np.array([[0, -1j], [1j, 0]], dtype=complex)
print("Pauli-Y matrix Y =\n", Y)
for label, state in [("|0>", ket0), ("|1>", ket1)]:
    c, q = run_gate(Y, state, qml.PauliY)
    print(f"Y . {label} (matrix)    = {c}")
    print(f"Y . {label} (PennyLane) = {q}")
```

Output:

```
Pauli-Y matrix Y =
[[ 0.+0.j -0.-1.j]
 [ 0.+1.j  0.+0.j]]
Y . |0> (matrix)    = [0.+0.j 0.+1.j]
Y . |0> (PennyLane) = [0.+0.j 0.+1.j]
Y . |1> (matrix)    = [0.-1.j 0.+0.j]
Y . |1> (PennyLane) = [0.-1.j 0.+0.j]
```

Observation: Pauli-Y flips the bit AND introduces an imaginary phase (+i or -i).

Program 4: Implement the Pauli-Z gate matrix and verify its phase inversion using matrix multiplication and PennyLane.

OUTPUT:

Code:

```
Z = np.array([[1, 0], [0, -1]], dtype=complex)
print("Pauli-Z matrix Z =\n", Z)
for label, state in [("|0>", ket0), ("|1>", ket1)]:
    c, q = run_gate(Z, state, qml.PauliZ)
    print(f"Z . {label} (matrix)    = {c}")
    print(f"Z . {label} (PennyLane) = {q}")
```

Output:

```
Pauli-Z matrix Z =
[[ 1.+0.j  0.+0.j]
 [ 0.+0.j -1.+0.j]]
Z . |0> (matrix)    = [1.+0.j 0.+0.j]
```



```
Z . |0> (PennyLane) = [ 1.+0.j -0.+0.j]
Z . |1> (matrix)      = [ 0.+0.j -1.+0.j]
Z . |1> (PennyLane) = [ 0.+0.j -1.+0.j]
```

Observation: Pauli-Z leaves $|0\rangle$ unchanged and inverts the phase of $|1\rangle$ (multiplies it by -1).

Program 5: Implement the Phase (S) gate matrix and verify its phase-shift operation on the computational basis states.

OUTPUT:

Code:

```
S = np.array([[1, 0], [0, 1j]], dtype=complex)
print("Phase (S) matrix S =\n", S)
for label, state in [("|0>", ket0), ("|1>", ket1)]:
    c, q = run_gate(S, state, qml.S)
    print(f"S . {label} (matrix)      = {c}")
    print(f"S . {label} (PennyLane) = {q}")
```

Output:

```
Phase (S) matrix S =
[[1.+0.j 0.+0.j]
 [0.+0.j 0.+1.j]]
S . |0> (matrix)      = [1.+0.j 0.+0.j]
S . |0> (PennyLane) = [1.+0.j 0.+0.j]
S . |1> (matrix)      = [0.+0.j 0.+1.j]
S . |1> (PennyLane) = [0.+0.j 0.+1.j]
```

Observation: The S gate leaves $|0\rangle$ unchanged and applies a $\pi/2$ phase shift (multiplies by i) to $|1\rangle$.

Program 6: Implement the T gate matrix and verify the $\pi/4$ phase rotation using matrix multiplication and PennyLane simulation.

OUTPUT:

Code:

```
T = np.array([[1, 0], [0, np.exp(1j*np.pi/4)]], dtype=complex)
print("T matrix T =\n", T)
print("e^(i*pi/4) =", np.exp(1j*np.pi/4))
for label, state in [("|0>", ket0), ("|1>", ket1)]:
    c, q = run_gate(T, state, qml.T)
    print(f"T . {label} (matrix)      = {c}")
    print(f"T . {label} (PennyLane) = {q}")
```

Output:

```
T matrix T =
[[1.          +0.j          0.          +0.j          ]
 [0.          +0.j          0.70710678+0.70710678j]]
e^(i*pi/4) = (0.7071067811865476+0.7071067811865475j)
T . |0> (matrix)      = [1.+0.j 0.+0.j]
T . |0> (PennyLane) = [1.+0.j 0.+0.j]
T . |1> (matrix)      = [0.          +0.j          0.70710678+0.70710678j]
T . |1> (PennyLane) = [0.          +0.j          0.70710678+0.70710678j]
```



Observation: The T gate leaves $|0\rangle$ unchanged and applies a $\pi/4$ phase rotation to $|1\rangle$.

Program 7: Implement the Hadamard gate matrix and verify the creation of the equal superposition state using analytical calculations and PennyLane simulation.

OUTPUT:

Code:

```
H = (1/np.sqrt(2)) * np.array([[1, 1], [1, -1]], dtype=complex)
print("Hadamard matrix H =\n", H)
for label, state in [("|0>", ket0), ("|1>", ket1)]:
    c, q = run_gate(H, state, qml.Hadamard)
    print(f"H . {label} (matrix)      = {c}")
    print(f"H . {label} (PennyLane) = {q}")
```

Output:

```
Hadamard matrix H =
[[ 0.70710678+0.j  0.70710678+0.j]
 [ 0.70710678+0.j -0.70710678+0.j]]
H . |0> (matrix)      = [0.70710678+0.j  0.70710678+0.j]
H . |0> (PennyLane) = [0.70710678+0.j  0.70710678+0.j]
H . |1> (matrix)      = [ 0.70710678+0.j -0.70710678+0.j]
H . |1> (PennyLane) = [ 0.70710678+0.j -0.70710678+0.j]
```

Observation: The Hadamard gate creates an equal superposition of $|0\rangle$ and $|1\rangle$, with a relative minus sign appearing when starting from $|1\rangle$.

Program 8: Compare the transformations produced by the Identity, Pauli-X, Pauli-Y, Pauli-Z, Phase (S), T, and Hadamard gates on the computational basis state $|0\rangle$.

OUTPUT:

Code:

```
gates_p8 = {
    "Identity": (I, qml.Identity), "Pauli-X": (X, qml.PauliX), "Pauli-Y": (Y,
    qml.PauliY),
    "Pauli-Z": (Z, qml.PauliZ), "Phase (S)": (S, qml.S), "T": (T, qml.T),
    "Hadamard": (H, qml.Hadamard),
}
results_p8 = {}
for name, (mat, op) in gates_p8.items():
    c, q = run_gate(mat, ket0, op)
    results_p8[name] = q
    print(f"{name:>10} . |0> = {q} (probabilities: {np.round(np.abs(q)**2,3)})")

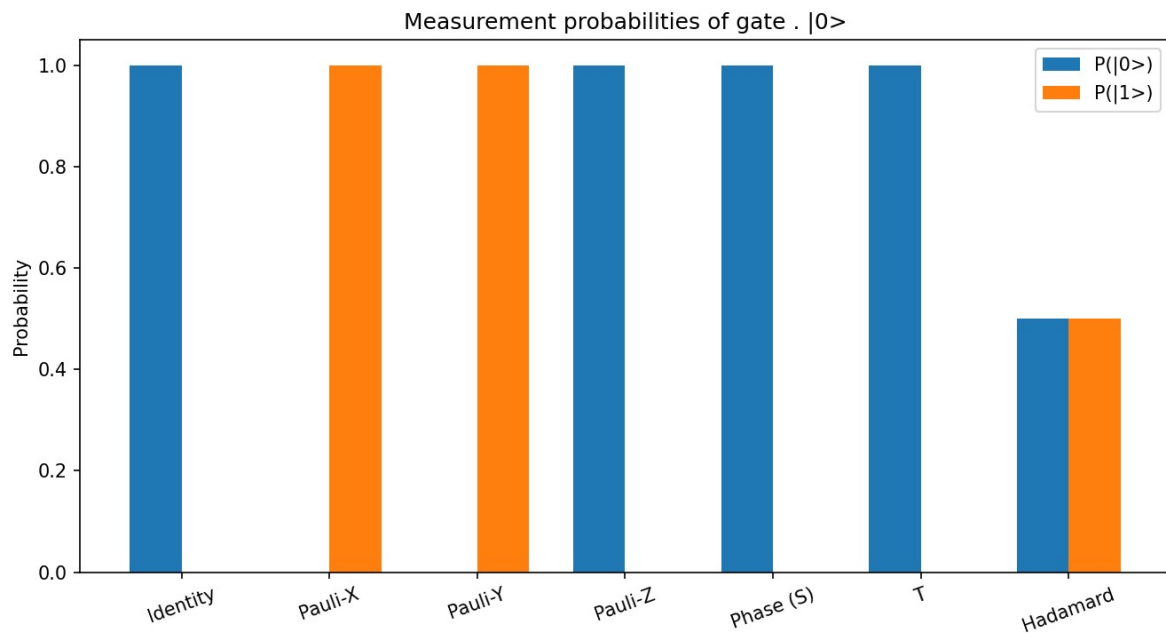
fig, ax = plt.subplots(figsize=(9,5))
names = list(results_p8.keys())
probs0 = [np.abs(results_p8[n][0])**2 for n in names]
probs1 = [np.abs(results_p8[n][1])**2 for n in names]
x = np.arange(len(names)); width = 0.35
ax.bar(x - width/2, probs0, width, label='P(|0>)')
ax.bar(x + width/2, probs1, width, label='P(|1>)')
ax.set_xticks(x); ax.set_xticklabels(names, rotation=20)
ax.set_ylabel('Probability'); ax.set_title('Measurement probabilities of gate . |
0>')
ax.legend(); plt.tight_layout(); plt.show()
```



Output:

```
Identity . |0> = [1.+0.j 0.+0.j] (probabilities: [1. 0.])
Pauli-X . |0> = [0.+0.j 1.+0.j] (probabilities: [0. 1.])
Pauli-Y . |0> = [0.+0.j 0.+1.j] (probabilities: [0. 1.])
Pauli-Z . |0> = [1.+0.j -0.+0.j] (probabilities: [1. 0.])
Phase (S) . |0> = [1.+0.j 0.+0.j] (probabilities: [1. 0.])
T . |0> = [1.+0.j 0.+0.j] (probabilities: [1. 0.])
Hadamard . |0> = [0.70710678+0.j 0.70710678+0.j] (probabilities: [0.5 0.5])
```

Observation: X and H move population toward/across $|1\rangle$, while Y, Z, S, T only change the phase relative to $|0\rangle$ (so their probabilities match $|0\rangle$ exactly); H splits the probability equally.



Program 9: Verify the unitary property of the Pauli-X, Pauli-Y, Pauli-Z, Phase (S), T, and Hadamard matrices by computing $U^\dagger U$.

OUTPUT:

Code:

```
gates_p9 = {"Pauli-X": X, "Pauli-Y": Y, "Pauli-Z": Z, "Phase (S)": S, "T": T,
"Hadamard": H}
for name, U in gates_p9.items():
    product = np.conj(U).T @ U
    is_unitary = np.allclose(product, I)
    print(f"{name:>10}: U-dagger . U =\n{np.round(product,3)}\n Unitary?
{is_unitary}\n")
```

Output:

```
Pauli-X: U-dagger . U =
[[1.+0.j 0.+0.j]
 [0.+0.j 1.+0.j]]
Unitary? True
```

```
Pauli-Y: U-dagger . U =
[[1.+0.j 0.+0.j]
```



```
[0.+0.j 1.+0.j]]
Unitary? True
```

```
Pauli-Z: U-dagger . U =
[[1.+0.j 0.+0.j]
 [0.+0.j 1.+0.j]]
Unitary? True
```

```
Phase (S): U-dagger . U =
[[1.+0.j 0.+0.j]
 [0.+0.j 1.+0.j]]
Unitary? True
```

```
T: U-dagger . U =
[[1.+0.j 0.+0.j]
 [0.+0.j 1.-0.j]]
Unitary? True
```

```
Hadamard: U-dagger . U =
[[ 1.+0.j -0.+0.j]
 [-0.+0.j 1.+0.j]]
Unitary? True
```

Program 10: Develop a PennyLane program to implement all the fundamental one-qubit quantum gates, compare their matrix representations with simulated quantum state transformations, and visualize the resulting probability distributions.

OUTPUT:

Code:

```
all_gates = {"I": (I, qml.Identity), "X": (X, qml.PauliX), "Y": (Y, qml.PauliY),
            "Z": (Z, qml.PauliZ),
            "S": (S, qml.S), "T": (T, qml.T), "H": (H, qml.Hadamard)}
final_states = {}
for name, (mat, op) in all_gates.items():
    c, q = run_gate(mat, ket0, op)
    final_states[name] = q
    match = np.allclose(c, q)
    print(f"Gate {name}: matrix result = {np.round(c,3)}, PennyLane result =
{np.round(q,3)}, match = {match}")

fig, axes = plt.subplots(1, len(all_gates), figsize=(16,3.2), sharey=True)
for ax, (name, state) in zip(axes, final_states.items()):
    probs = np.abs(state)**2
    ax.bar(['|0>', '|1>'], probs, color=['#4C72B0', '#DD8452'])
    ax.set_title(name); ax.set_ylim(0,1.05)
fig.suptitle('Probability distributions of all fundamental 1-qubit gates acting on
|0>')
plt.tight_layout(); plt.show()
```

Output:

```
Gate I: matrix result = [1.+0.j 0.+0.j], PennyLane result = [1.+0.j 0.+0.j], match
= True
Gate X: matrix result = [0.+0.j 1.+0.j], PennyLane result = [0.+0.j 1.+0.j], match
= True
Gate Y: matrix result = [0.+0.j 0.+1.j], PennyLane result = [0.+0.j 0.+1.j], match
= True
Gate Z: matrix result = [1.+0.j 0.+0.j], PennyLane result = [ 1.+0.j -0.+0.j],
```



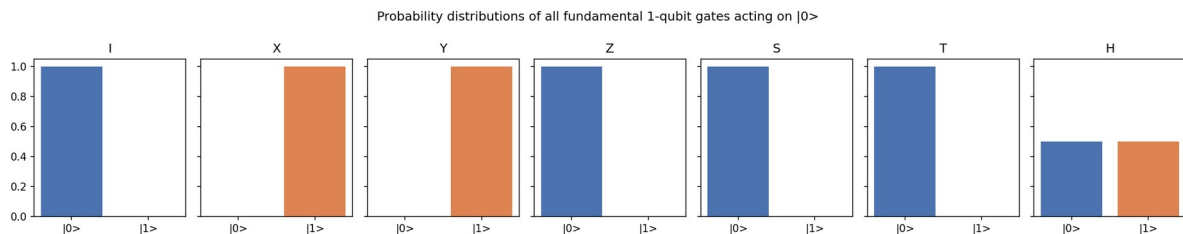
match = True

Gate S: matrix result = $[1.+0.j \ 0.+0.j]$, PennyLane result = $[1.+0.j \ 0.+0.j]$, match = True

Gate T: matrix result = $[1.+0.j \ 0.+0.j]$, PennyLane result = $[1.+0.j \ 0.+0.j]$, match = True

Gate H: matrix result = $[0.707+0.j \ 0.707+0.j]$, PennyLane result = $[0.707+0.j \ 0.707+0.j]$, match = True

Observation: All matrix-based and PennyLane-simulated results match exactly, confirming correct implementation of every fundamental single-qubit gate.



VIVA Q&A:

1. What is PennyLane?
2. Why is PennyLane used in quantum computing?
3. What is a QNode?
4. What is a quantum device in PennyLane?
5. Which function is used to create a QNode?
6. How do you apply a quantum gate in PennyLane?
7. What is the purpose of `qml.device()`?
8. What is an expectation value?
9. How is a quantum circuit executed in PennyLane?
10. What are the applications of PennyLane?

RESULT:

The fundamentals of the PennyLane quantum computing framework were studied and verified. Quantum devices and QNodes were created, and the Identity, Pauli-X, Pauli-Y, Pauli-Z, Phase (S), T, and Hadamard single-qubit gates were implemented, applied to the computational basis states, and simulated successfully. The matrix multiplication results matched the PennyLane simulation results exactly in every program, all gate matrices were verified to be unitary ($U^\dagger U = I$), and the resulting probability distributions were visualized. Thus, the implementation of fundamental single-qubit quantum gates using PennyLane was executed and verified successfully.